

Observer

Patrón Observer

1. Explicación del patrón

Observer es un patrón de comportamiento que define una relación de uno a muchos entre objetos.

La idea principal es que existe un objeto principal, llamado sujeto o notificador, que mantiene un estado importante. Otros objetos, llamados observadores o suscriptores, están interesados en los cambios de ese sujeto.

Cuando el sujeto cambia de estado, notifica automáticamente a todos sus observadores para que puedan actualizarse.

Esto permite mantener la consistencia entre objetos relacionados sin crear un acoplamiento fuerte entre ellos. El sujeto no necesita conocer las clases concretas de sus observadores, solo necesita saber que cumplen una interfaz común de actualización.

También puede entenderse como un mecanismo de suscripción:

- Los observadores se suscriben al sujeto.
- El sujeto guarda una lista de observadores.
- Cuando ocurre un cambio relevante, el sujeto recorre esa lista y avisa a todos.
- Los observadores pueden actualizarse en respuesta a esa notificación.

2. Cuándo se utiliza

Se utiliza el patrón Observer cuando:

- Un objeto cambia de estado y otros objetos deben enterarse automáticamente.
- No se sabe de antemano cuántos objetos tendrán que actualizarse cuando ocurra un cambio.
- Se quiere que un objeto notifique a otros sin depender de sus clases concretas.
- Se busca bajo acoplamiento entre el objeto que genera el cambio y los objetos que reaccionan.
- Una abstracción tiene dos aspectos relacionados y uno depende del otro.
- Se quiere separar la lógica principal de una aplicación de las partes que reaccionan a sus cambios.
- Algunos objetos deben observar a otros solo durante cierto tiempo o en situaciones concretas.
- La lista de objetos interesados puede cambiar dinámicamente durante la ejecución.

Un caso típico es una interfaz gráfica: los datos de la aplicación cambian y varias vistas deben actualizarse automáticamente, por ejemplo una tabla, un gráfico o varios paneles de información.

3. Cómo aplicarlo

Para aplicar Observer se suelen definir estos participantes:

Sujeto

Es el objeto observado.

Responsabilidades:

- Mantener una lista de observadores.
- Permitir añadir observadores.
- Permitir eliminar observadores.
- Notificar a los observadores cuando ocurre un cambio.

Operaciones habituales:

- `agregarObservador(observador)`
- `eliminarObservador(observador)`
- `notificarObservadores()`

Observador

Es la interfaz común que deben implementar todos los objetos que quieran recibir notificaciones.

Responsabilidad principal:

- Definir el método de actualización.

Operación habitual:

- `actualizar()`

En algunos diseños, el método `actualizar()` puede recibir información contextual sobre el cambio, por ejemplo el propio sujeto, el nuevo estado o el evento producido.

Sujeto concreto

Es una implementación concreta del sujeto.

Responsabilidades:

- Guardar el estado que interesa a los observadores.
- Modificar su estado.
- Avisar a los observadores cuando el estado cambia.

Operaciones habituales:

- `getEstado()`
- `setEstado()`
- `modificarEstado()`
- `notificarObservadores()`

Observador concreto

Es una clase concreta que reacciona ante los cambios del sujeto.

Responsabilidades:

- Mantener, si lo necesita, una referencia al sujeto concreto.
- Guardar el estado del sujeto que le interesa.
- Implementar el método `actualizar()` .
- Mantener su estado consistente con el del sujeto.

4. Forma práctica de aplicarlo

Pasos:

1. Detectar qué objeto tiene el estado importante.
2. Convertir ese objeto en el sujeto o notificador.
3. Crear una interfaz `Observador` con un método `actualizar()` .
4. Hacer que todos los objetos interesados implementen esa interfaz.
5. Añadir al sujeto una lista de observadores.
6. Añadir métodos para suscribir y desuscribir observadores.
7. Cada vez que el sujeto cambie de estado, llamar a `notificarObservadores()` .
8. Dentro de `notificarObservadores()` , recorrer la lista de observadores y llamar a `actualizar()` en cada uno.
9. En cada observador concreto, implementar la reacción concreta al cambio.

5. Idea clave

El sujeto no debe depender de observadores concretos.

Solo debe conocer una interfaz común.

Así, se pueden añadir nuevos observadores sin modificar el sujeto.

6. Ejemplos del material

Ejemplo de sensor

Un sensor puede actuar como sujeto.

Cuando salta una alarma, el sensor notifica a todos sus observadores.

Observadores posibles:

- Policía
- Protección Civil
- Bomberos

Cada uno implementa su propia forma de reaccionar mediante `actualizar()` .

Ejemplo de partido de fútbol

Un partido de fútbol puede actuar como sujeto.

Cuando ocurre un evento importante, como un gol o el avance del tiempo, notifica a sus observadores.

Observadores posibles:

- Prensa escrita
- Radio
- TV

Cada observador se actualiza de una forma distinta:

- La prensa publica una crónica.
- La radio emite un comentario.
- La TV muestra la actualización en pantalla.

7. Ventajas prácticas

- Reduce el acoplamiento entre el sujeto y los observadores.
- Permite añadir nuevos observadores sin modificar el sujeto.
- Permite eliminar observadores cuando ya no sean necesarios.
- Facilita mantener varios objetos sincronizados.
- Permite crear relaciones dinámicas entre objetos durante la ejecución.
- Separa el objeto que produce el cambio de los objetos que reaccionan a él.

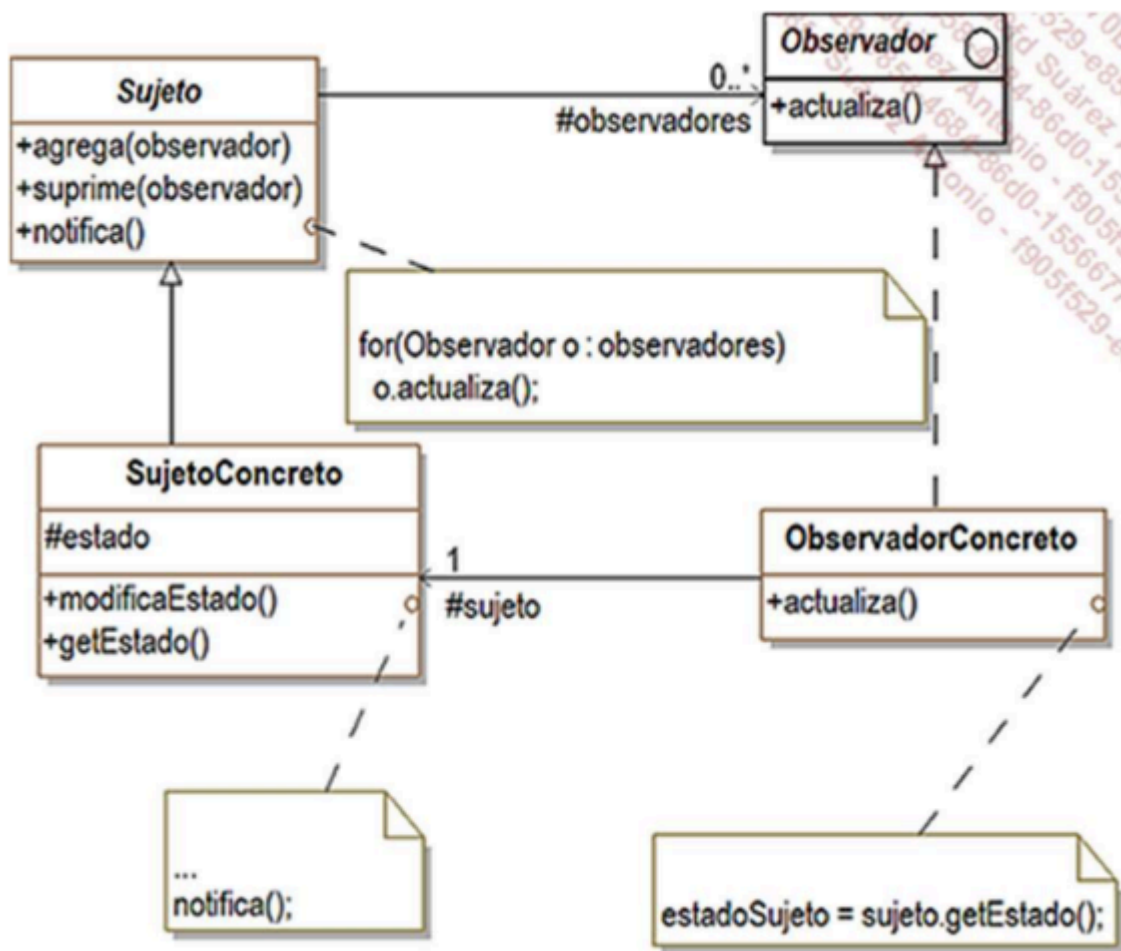
8. Cuidado

El orden de notificación de los observadores no siempre debe ser relevante.

Si el comportamiento depende del orden en que se actualizan los observadores, el diseño puede volverse más frágil.

También hay que evitar que los observadores queden suscritos cuando ya no deben recibir notificaciones, porque podrían seguir reaccionando a cambios que ya no les corresponden.

9. Hueco para la plantilla



 I_Observador

```
public interface Observador {  
    public void actualizar(String accion, String lugar);  
}
```



```
public class SujetoConcreto implements Sujeto{

    private String accion;
    private String lugar;

    private ArrayList<Observador> observadores;

    public SujetoConcreto(){
        observadores = new ArrayList<Observador>();
    }

    @Override
    public void registrarObservador(Observador o) {
        observadores.add(o);
    }

    @Override
    public void retirarObservador(Observador o) {
        observadores.remove(o);
    }

    @Override
    public void ejecutaAccion(String accion, String lugar) {
        this.accion = accion;
        this.lugar = lugar;
        notifica();
    }

    @Override
    public void notifica() {
        for(Observador o:observadores){
            o.actualizar(accion, lugar);
        }
    }

}
```



```
public interface Sujeto {  
    public void registrarObservador(Observador o);  
    public void retirarObservador(Observador o);  
  
    public void ejecutaAccion(String accion, String lugar);  
  
    public void notifica();  
}
```



```
public class ObserverPlantilla {  
  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        Sujeto sujetoConcreto = new SujetoConcreto();  
        Observador obser = new ObservadorConcreto();  
        Observador obser2= new ObservadorConcreto2();  
        String accion;  
        String lugar;  
  
        System.out.println("Registrar los observadores");  
        sujetoConcreto.registrarObservador(obser);  
        sujetoConcreto.registrarObservador(obser2);  
  
        System.out.println("Indique accion: ");  
        accion = sc.nextLine();  
  
        System.out.println("Indique un lugar");  
        lugar = sc.nextLine();  
  
        sujetoConcreto.ejecutaAccion(accion, lugar);  
    }  
  
}
```



```
public class ObservadorConcreto2 implements Observador{  
  
    @Override  
    public void actualizar(String accion, String lugar) {  
        System.out.println("Soy observador concreto2 con la accion " + accion + " en el lugar " + lugar);  
    }  
  
}
```

```
ObservadorConcreto

public class ObservadorConcreto implements Observador{

    @Override
    public void actualizar(String accion, String lugar) {
        System.out.println("Soy observador concreto con la accion " + accion + " en el lugar " + lugar);
    }

}
```

codesnap.dev

10. Hueco para el ejemplo

No se encontró "Pasted image 20260516183908.png".

```
Bomberos

public class Bomberos implements IObservador{

    @Override
    public void actualiza() {
        System.out.println("Bomberos recibe alarma");
    }

}
```

codesnap.dev



```
public class Sensor implements ISensor{

    private ArrayList<IObservador> observadores;

    public Sensor(){
        observadores = new ArrayList<IObservador>();
    }

    @Override
    public void saltarAlarma(String tipo) {
        System.out.println("Hay una alarma de tipo " + tipo);
        notificaObservadores();
    }

    @Override
    public void agregarObservador(IObservador o) {
        observadores.add(o);
    }

    @Override
    public void eliminarObservador(IObservador o) {
        observadores.remove(o);
    }

    @Override
    public void notificaObservadores() {
        for(IObservador o:observadores){
            o.actualiza();
        }
    }

}
```



ProteccionCivil

```
public class ProteccionCivil implements IObservador{

    @Override
    public void actualiza() {
        System.out.println("Proteccion civil recibe alarma");
    }

}
```

codesnap.dev



Policia

```
public class Policia implements IObservador{

    public Policia(){

    }

    @Override
    public void actualiza() {
        System.out.println("Policia recibe alarma");
    }

}
```

codesnap.dev

```
public class ObserverAlarma {  
  
    public static void main(String[] args) {  
        ISensor casa = new Sensor();  
        IObservador policia = new Policia();  
        IObservador bomberos = new Bomberos();  
        IObservador proteccionCivil = new ProteccionCivil();  
  
        casa.agregarObservador(policia);  
        casa.agregarObservador(bomberos);  
        casa.agregarObservador(proteccionCivil);  
        casa.saltarAlarma("INCENDIO");  
  
        casa.eliminarObservador(bomberos);  
        casa.eliminarObservador(proteccionCivil);  
        casa.saltarAlarma("OKUPA");  
  
    }  
  
}
```

```
public interface ISensor {  
    public void saltarAlarma(String tipo);  
  
    public void agregarObservador(IObservador o);  
    public void eliminarObservador(IObservador o);  
    public void notificaObservadores();  
  
}
```



I_Observador

```
public interface IObservador {  
    public void actualiza();  
  
}
```

codesnap.dev